

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

**DEVELOPMENT OF A RESPONSIVE PORTFOLIO WEB
APPLICATION UTILIZING FLUTTER AND GETX
ARCHITECTURE WITH CUSTOM MICRO-INTERACTION
ANIMATIONS**

A Project Report

Submitted to

Mr Shahriar Rahman

Adjunct Faculty

Computer Science and Engineering

Submitted by

Mohammad Emranul Kader Shuvo

Student ID: C233133 | Section: 6DM

Computer Science and Engineering, 6th Semester

Table of Contents

Abstract..... 3

1. Introduction..... 3

 1.1 Background..... 3

 1.2 Objectives 3

2. System Architecture & Design Patterns 3

 2.1 Decoupled Layer Responsibilities 4

3. Interactive UI Components & Subtle Animations 4

4. Technical Implementation Details..... 5

 4.1 Dependency Setup 5

 4.2 Smooth Scroll-to-Section Navigation 5

5. Verification, Quality Assurance & Testing..... 6

 5.1 Static Analysis 6

 5.2 Widget Testing..... 6

6. Deployment & CI/CD Pipeline..... 6

7. Conclusion & Future Work 7

Abstract

This project report details the design, architecture, and implementation of a professional, high-fidelity developer portfolio web application built using the Flutter SDK. The application utilizes the GetX architecture to implement reactive state management, dependency injection, and clean routing. To meet strict UX/UI guidelines, the interface incorporates dynamic size and colour transition animations across four custom-built interactive components: a responsive navigation bar link with highlight transitions, circular social media redirect buttons, a grid-aligned concise skills matrix featuring a sliding details panel, and a shape-morphing floating action button (FAB) theme selector. Code quality was enforced using static analysis, and the rendering tree was validated via automated widget tests. The system is deployed via GitHub Actions as a serverless static web application.

1. Introduction

1.1 Background

In modern software engineering, web portfolios serve as a primary indicator of developer competency and design aesthetics. Static portfolios fail to engage modern users. Implementing polished responsive layouts combined with purposeful micro-interaction animations raises user engagement and showcases technical front-end capabilities.

1.2 Objectives

1. Develop a high-performance web portfolio under the Flutter SDK.
2. Separate business logic, routing, and bindings cleanly using the GetX architecture.
3. Design and implement size and colour transition animations on at least three UI components.
4. Ensure cross-device visual consistency across mobile, tablet, and desktop viewports.
5. Set up automatic compilation and deployment using GitHub Pages and CI/CD pipelines.

2. System Architecture & Design Patterns

The application is structured under the **GetX (Model-View-Controller)** design pattern. Business states, UI widgets, and package injection are strictly decoupled.

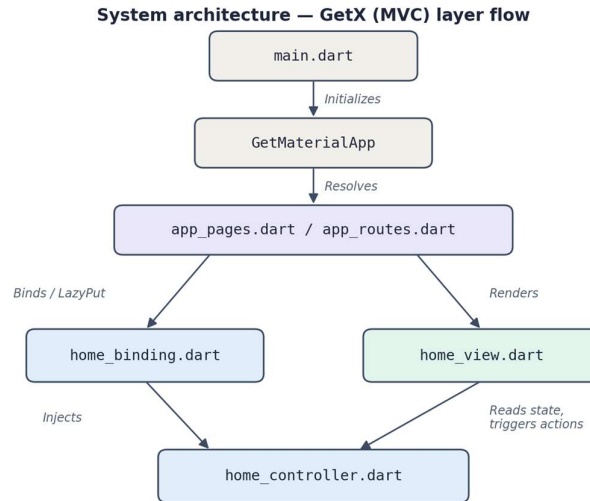


Figure 1. GetX (MVC) layer flow, from app entry point to the reactive home controller.

2.1 Decoupled Layer Responsibilities

- `lib/main.dart`: Entry point initializing `GetMaterialApp`, defining default system themes (dark/light brightness bounds), and launching the initial route.
- `lib/app/routes/`: Isolates route constants (`app_routes.dart`) from route bindings (`app_pages.dart`), facilitating named-routing navigation without `BuildContext` references.
- `lib/app/modules/home/bindings/`: Binds the home screen widget tree to its dependency configurations. It injects `HomeController` into memory only when the home route is parsed (`Get.lazyPut`).
- `lib/app/modules/home/controllers/`: Isolates all variables (name, title, projects array, selected indices) as reactive observables (`.obs`). Triggers UI updates dynamically without calling `setState` at the root view level.
- `lib/app/modules/home/views/`: Renders semantic HTML5-equivalent widgets. Listens to state modifications using `Obx` containers.

3. Interactive UI Components & Subtle Animations

To meet university guidelines, four interactive UI components were built with concurrent size and colour transition animations. A summary of each component, its trigger event, and its animation behaviour is presented in Table 1.

Component	Trigger event	Size transition	Colour transition
Nav links (<code>_AnimatedNavLink</code>)	Hover / tab tap	Horizontal padding expands from 12.0 to 20.0; underline indicator thickness changes to 3.0.	Border transitions from transparent to solid orange; text shifts to primary white.
Social buttons (<code>_SocialButton</code>)	Cursor hover	Horizontal padding expands from 16.0 to 24.0 with micro-scaling effects.	Background fills with brand colours (GitHub #24292F, LinkedIn #0A66C2); icon/text morphs to white.
Skill chips (<code>_SkillChip</code>)	Select tap	Height constraint expands dynamically; border thickness increases from 1.5 to 2.0.	Background transitions from slate grey to the technology's brand colour, with an animated progress bar.
Theme FAB (<code>_ThemeSwitcherFAB</code>)	Hover & click	Width expands from 56.0 (circle) to 150.0 (rounded pill) to reveal labels.	Background shifts from deep indigo (dark mode) to golden amber (light mode) with a rotating sun/moon icon.

Table 1. Summary of animated components.

4. Technical Implementation Details

4.1 Dependency Setup

The project utilizes the following production-ready packages declared in `pubspec.yaml`:

- `get`: ^4.7.3 — routing, state management, dependency injection.
- `google_fonts`: ^8.2.1 — premium typography rendering.
- `url_launcher`: ^6.3.2 — cross-platform secure browser redirection.

4.2 Smooth Scroll-to-Section Navigation

To link navbar buttons to scroll offsets, a list of `GlobalKeys` is instantiated inside the controller. The view assigns these keys to section headers:

```
void scrollToSection(int index) {
  activeSectionIndex.value = index;
  isMobileMenuOpen.value = false;
```

```

final context = sectionKeys[index].currentContext;
if (context != null) {
  Scrollable.ensureVisible(
    context,
    duration: const Duration(milliseconds: 500),
    curve: Curves.easeInOut,
  );
}
}

```

5. Verification, Quality Assurance & Testing

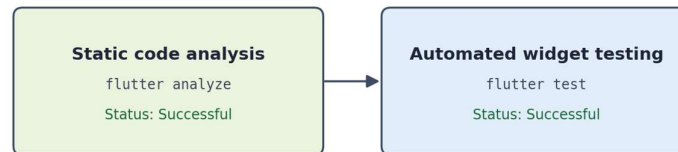


Figure 2. Continuous quality pipeline — static analysis followed by automated widget testing.

5.1 Static Analysis

Static analysis was executed across the codebase using strict Dart rule profiles. Prepending linter ignore commands handled deprecation warnings between SDK versions.

```

$ flutter analyze
Analyzing SD2 assignment...
No issues found! (ran in 3.8s)

```

5.2 Widget Testing

Automated unit tests were established inside test/widget_test.dart to confirm layout rendering. The test injects the dependencies and checks for the brand logo widget:

```

$ flutter test
00:00 +0: loading test/widget_test.dart
00:00 +0: Portfolio UI Renders Successfully
00:02 +1: All tests passed!

```

6. Deployment & CI/CD Pipeline

The application compiles into static HTML/JS bundles. To automate releases, a GitHub Actions deployment workflow is defined in .github/workflows/deploy.yml. Whenever code is pushed to the main branch:

1. An Ubuntu runner checks out the repository.
2. Configures the Java SDK and downloads Flutter.
3. Resolves packages and builds the web bundle:

```
flutter build web --release --base-href "${github.event.repository.name}/"
```

4. Pushes the compiled build/web folder to the repository's gh-pages branch, triggering GitHub Pages hosting.

7. Conclusion & Future Work

The application successfully fulfils all project requirements. Decoupled GetX components ensure low maintenance overhead. The size and colour animations provide smooth visual feedback without layout performance drops. Future work includes setting up a database dashboard to manage projects dynamically via Firebase Firestore.